

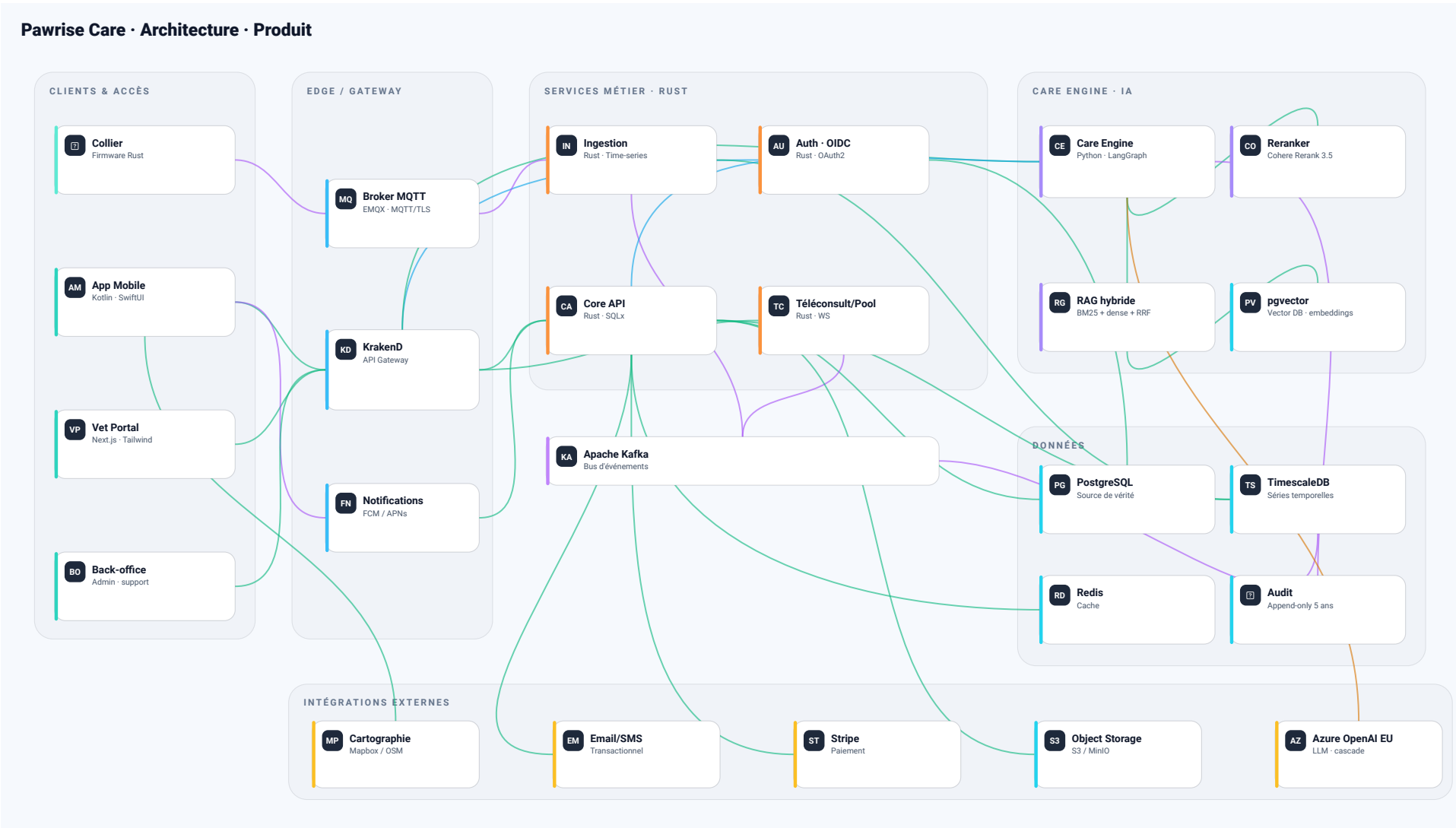
1 Architecture technique

1.1 Vue d'ensemble

L'architecture suit **quatre principes directeurs**, chacun assumé et justifié plus bas :

1. **Monolithe modulaire d'abord, macroservices ensuite** : un socle unique (API + Care Engine) pour la vélocité d'une équipe de 10, découpé en services indépendants seulement aux points chauds réels.
2. **Architecture événementielle** : les services communiquent via un bus (Kafka) pour se découpler et ne perdre aucun événement (ingestion, projections, alertes, audit, file de téléconsultation).
3. **IA en cage** : le Care Engine est un pipeline borné qui explique et oriente, sans jamais diagnostiquer ni écrire directement dans les données métier.
4. **Souveraineté et coût maîtrisé** : Rust économe, une seule famille de bases (PostgreSQL), hébergement UE auto-géré.

Flux : — synchrone (aller-retour) · — événement (sens unique) · — contrôle · — externe.



Plateforme & Ops (transverse) : Kubernetes auto-géré (Hetzner, Terraform) · GitOps (Argo CD) · observabilité OpenTelemetry → Prometheus / Loki / Tempo / Grafana · secrets chiffrés (SOPS). Détail en partie Cloud & infrastructure. Diagramme exporté du cockpit interactif : version complète et navigable sur pawrise-care.com.

Lecture par zone

- **Clients & accès** : une **app mobile native** (bien-être, localisation, alertes, chat IA, appairage BLE), un **Vet Portal web** (historique structuré + PDF normalisé) et une console **Admin**. Tout le trafic applicatif passe par **KrakenD**, une passerelle unique : un seul point d'entrée à sécuriser, qui centralise authentification, limitation de débit et routage.
- **Entrée IoT** : le **collier** parle en **MQTT sur TLS** (certificat par appareil) à un broker **EMQX**, séparé de la passerelle HTTP. MQTT est le standard IoT : pub/sub léger, QoS et mode hors-ligne gratuits, adaptés à une batterie et un réseau cellulaire instable.
- **Services métier (Rust)** : **OIDC** (identité et droits owner / vétérinaire), **Core API** (comptes, animaux, alertes), **Ingestion** (normalisation de la télémétrie), **véto & abonnement**. Ils forment d'abord un **monolithe modulaire** : on n'extrait un macroservice que lorsqu'un besoin réel (charge, équipe) le justifie.
- **Bus d'événements (Kafka)** : la colonne vertébrale. Les services **publient** des événements ; les consommateurs (projections, Care Engine, audit, file de téléconsultation) s'y **abonnent**. Résultat : services découplés et **aucun événement perdu**.
- **IA · Care Engine (Python)** : un pipeline **LangGraph borné** qui consomme le **RAG** (pgvector) et le LLM (**Azure OpenAI UE** + reranker Cohere). Il **ne peut pas écrire** dans les données métier : il produit une orientation et, si besoin, **escalade** vers le vétérinaire via une interface contrôlée.
- **Données** : une seule famille **PostgreSQL** : transactionnel (PostgreSQL), séries temporelles de télémétrie (**TimescaleDB**) et embeddings du corpus (**pgvector**), plus **Redis** pour le cache et les sessions. Un seul moteur à opérer, sauvegarder et sécuriser.

L'IA EN CAGE · RÈGLE D'OR

Le Care Engine n'a **aucun accès en écriture** au métier et suit un pipeline déterministe : il explique, oriente et escalade, **jamais il ne diagnostique** (Code rural, art. L243-1). Cette contrainte est portée par l'architecture elle-même, pas seulement par le modèle, ce qui la rend juridiquement défendable (ADR-001).

1.2 Choix techniques justifiés

Chaque décision structurante : besoin, alternatives écartées, décision, et trade-off assumé.

Choix	Alternatives écartées	Décision & trade-off
Rust (services métier)	Node, Python, Go, C#	Perf + sûreté mémoire, empreinte minimale. Coût dev ~2,5× assumé (mitigé : monolithe-first, Python pour l'IA, réintroduction ciblée).
MQTT / EMQX (entrée IoT)	HTTP, CoAP, passerelle maison	Standard IoT : pub/sub léger, QoS, offline gratuits. Un composant de plus à opérer.
Apache Kafka (bus)	RabbitMQ, appels directs	Débit, rétention, découplage. Plus gourmand en RAM (dimensionne le cluster).
PostgreSQL + TimescaleDB + pgvector	Mongo + Influx + Qdrant séparés	Une seule famille : transactionnel, séries temporelles, vectoriel. Bascule Qdrant possible.
Pipeline LangGraph borné	Agent LLM autonome	Chaque étape testable, responsabilité juridique portée par le pipeline (ADR-001).
Azure OpenAI EU	LLM hors UE, modèle unique	Data residency UE, ~0,03-0,05 €/conv. Dépendance Microsoft mitigée (interface LLMPProvider).

Choix	Alternatives écartées	Décision & trade-off
+ cascade + Cohere		
Kubernetes auto-géré + GitOps	PaaS, managé d'emblée	Économique et souverain ; plus d'ops, mitigé par l'IaC. Managé EU au scale.
OIDC maison	Auth0, Keycloak, Cognito	Souveraineté + défi formateur ; coûteux (~4-6 mois-homme), revue de sécurité obligatoire.